Automatically defines Constructor's default values → Canonical Constructor

Custom Constructor

overdefine ()

Compact for

```
public Rem
{  if( he > to )
   {
   }
}
```

→ Compact fo

You don't specify parameters → Compact form

You cannot read or modify instance field → Compact

What if we need to store not update data

JDK - 16

Records

private final

record point (double x, double y)
{

}

automatically creates private final getters

x()
y()

It is legal in java accessors & instance variable name.

record → static fields, methods

You cannot add instance fields ✗

But final → mutable

It means only we can refer to new object

Records

Data → just data

Data hiding oop — away

Point (x,y)

```
class Point {
    private final double x;
    private final double y;
    public Point(double x, double y)
    { this.x = x;
      this.y = y;
    }
    public double getX()
    { return x;
    }
    public double getY()
    { return y;
    }
    public String toString()
    { return "Point[x=" +
    }
}
```

Object Destruction

Automatic memory management

Automatic Garbage Collection (Memory)

File ← some objects

↓

close() method to clean the resources

Runtime.addShutDownHook → virtual machine to Shutdown

Cleaner → Java 9

finalize() → ~~finalize~~ deprecated

Static {

}

↓

Every time class loads

Order of execution same as order of declaration

→ Static also get default value

Constructor Execution

first()

↓

second()

↓

instance fields → default values

↓

field initialization blocks order class wise

↓

constructor execution

Calling another Constructor

this( ) -> first statement of constructor of same class

Calling Constructor of same class

Constructor chaining :-

Initialization Blocks

```
class Emp{
    int id;
    Emp(int id)
    { this.id=id;
    }
    Emp()
    { this(10);
    }
}
```

Initialize values

1- By creating a value in constructor

2- By giving a value in block

```
{
}
```

{} -> blocks initialization blocks

Which executes in order every time object construct

Parameter Names

```
class Emp
{   int id;
    String name;
    public Emp(int id, String name)
    {   this.name = name;
        this.id = id;
    }
}
```

Variable shadowing

Local variable → High priority

Just hides the instance ones

this → Explicit parameter

```
this.name = name;
this.id = id;
```

objects should be constructed
just like the buildings are constructed

Default constructor with no args

different inputs → overloads

Explicit field Initialization

You can also pass fields explicitly inside constructor
(Parametrized Constructors)

```
class Emp { int id;
  Emp()
  {
  }
  Emp(int id)
  { this.id = id;
  }
}
```

→ Constructor overloading

default

parametrized

Default field Initialization

- Static & Instance
  - number → 0
  - boolean → false
  - object reference → null

local variables ✗ not get default value

must initialize

Constructor with No Arguments

- ☆ Set Initial values
- ☆ Default constructor

If you don't explicitly provide a constructor, compiler provides it for you.

But when you have explicitly written constructor

Compiler won't provide anything

object constructors

Constructors

↓

They construct objects

Initialize values

Overloading

Some class more than one constructor → Constructor overloading

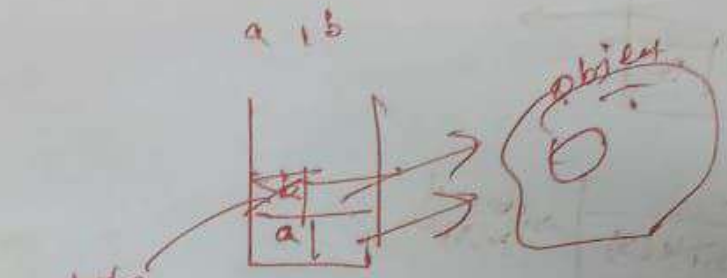
new
new StringBuilder();
StringBuild ("a");

Compiler → sort out method or constructor right number of parameters

else error

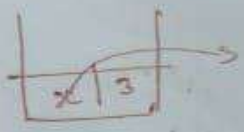
Compiler cannot found → error

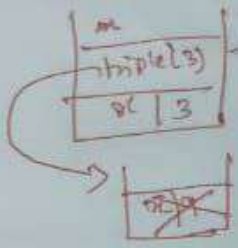
Overloading resolution



after method ends

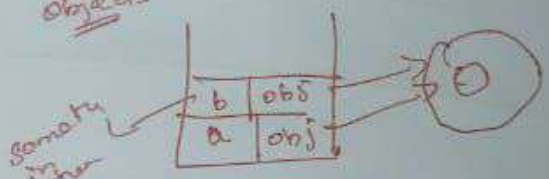
Method cannot make refer to new object





But when method ends
x = 8

Objects



They refer to same object
That's why they can change state
But try to swap don't work ✗

That it's the call by value

Java call by value